OS202 - Systèmes Parallèles et Distribués

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

Gustavo DE MACENA BARRETO
Luiza GONÇALVES SOARES

Encadrant : Augustin PARRET-FRÉAUD
Responsable matière : Xavier JUVIGNY

École Nationale des techniques avancées (ENSTA Paris)
Institut Polytechnique de Paris
Mars 2026

1. Introduction

Définition du Problème

Le défi central de l'optimisation par colonie de fourmis sur un paysage fractal réside dans la résolution du scénario de fourragement. Dans ce modèle, une population d'agents simples doit identifier le chemin le plus court entre la fourmilière et une source de nourriture sur un terrain représenté par une grille cartésienne 2D.

Cet environnement se caractérise par une carte d'unités de temps générée de façon stochastique (plasma fractal), où chaque cellule possède une valeur comprise entre zéro et un, traduisant la difficulté de traversée. Les fourmis sont des agents réactifs qui communiquent indirectement avec leur environnement par la mise à jour de deux types de phéromones :

— Le phéromone $V_1(s)$, permettant d'indiquer aux autres fourmis le chemin d'exploration effectué.
— Le phéromone $V_2(s)$, permettant à la fourmi de retourner au nid après avoir trouvé de la nourriture.

Objectifs et Méthodologie d'Optimisation

Pour résoudre ce problème de manière efficace sous l'angle du calcul haute performance, le projet exige la transformation du modèle séquentiel en une application optimisée à travers les étapes suivantes :

— **Mesure du temps passé par itération** : mesurer et noter le temps passé dans les différentes parties du code original pour établir une base de comparaison.
— **Vectorisation des algorithmes** : restructurer le code en utilisant des tableaux pour stocker les coordonnées, les états et les graines de toutes les fourmis, permettant de repérer chaque agent uniquement par son indice.
— **Parallélisation en mémoire partagée** : utiliser l'interface **OpenMP** pour paralléliser les boucles de calcul critiques afin d'obtenir une amélioration nette des performances.
— **Parallélisation distribuée** : concevoir et évaluer des stratégies de division du travail, soit en répartissant les fourmis entre des processus détenant la carte entière, soit en décomposant la carte en sous-zones gérées par différents processus pour optimiser l'occupation mémoire.

L'efficacité des techniques appliquées sera validée par une analyse du temps d'exécution et le calcul de l'accélération (*speedup*) obtenue à chaque phase par rapport à la version d'origine.

2. Analyse de Profilage Pré-vectorisation

Avant de l'optimisation du code, une analyse approfondie de la version séquentielle a été réalisée pour identifier les goulots d'étranglement structurels.

Analyse de l'algorithme

Dans cette section, nous réalisons une analyse du code initial, qui nous permet d'extraire des informations sur les goulets d'étranglement du processus. Dans la version initiale, nous avons

effectué le *profiling* du code afin d'évaluer les points critiques ainsi que les fonctions les plus coûteuses en temps de calcul.

Ce type d'analyse nous permet d'observer la contribution individuelle de chaque fonction par rapport au temps total d'exécution de l'algorithme. Dans une première observation, à l'aide de l'outil *perf-tools*, nous obtenons un graphique qui mesure la participation relative de chaque appel de fonction dans le temps d'exécution total.

Grâce à cet outil, nous avons identifié plusieurs points clés importants pour l'étape suivante de parallélisation — comme présenté dans la Figure 1 :

- La fonction `advance` représente environ 50% du temps total d'exécution.
- Le coût de l'affichage apparaît à travers les appels internes de la bibliothèque SDL, qui représentent environ 45% du coût total.
- Enfin, les 5% restants correspondent à des appels système liés à des défauts de page (*page faults*).

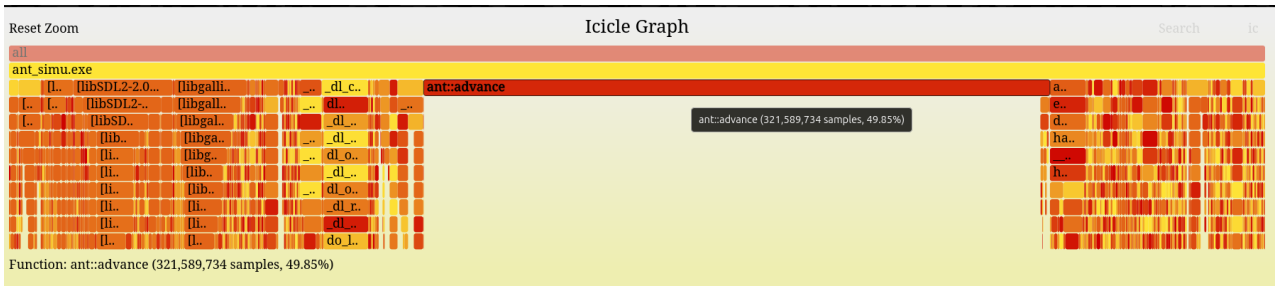

FIGURE 1 – Participation au temps d'exécution.

À partir de ces informations, et sachant que le système est organisé selon un modèle AoS (*Array of Structures*), l'accès mémoire dans cette architecture devient inefficace en raison du manque de contiguïté des données. Cela entraîne des retards dans la récupération des données depuis la mémoire, ce qui peut être atténué en modifiant l'architecture des données vers un modèle SoA (*Structure of Arrays*).

3. Vectorisation et openMP

Hiérarchie des Fonctions et Domaines de Calcul

La seconde analyse présente la répartition du temps de calcul par couches fonctionnelles. Le temps total est principalement divisé entre le rendu graphique et le moteur de simulation, comme on peut l'observer dans la Figure 2 :

- **Renderer : :display** : occupe environ 26,04% du temps, correspondant à l'affichage de la grille et des phéromones.
- **advance_time** : constitue le cœur du calcul avec 66,20% du temps total. Cette fonction se décompose en deux modules critiques :
  - *Mouvement* (`advance`) : gestion du déplacement des fourmis et dépôt de phéromones.
  - *Environnement* (`do_evaporation`) : mise à jour globale des taux de phéromones V_1 et V_2 sur toute la grille.

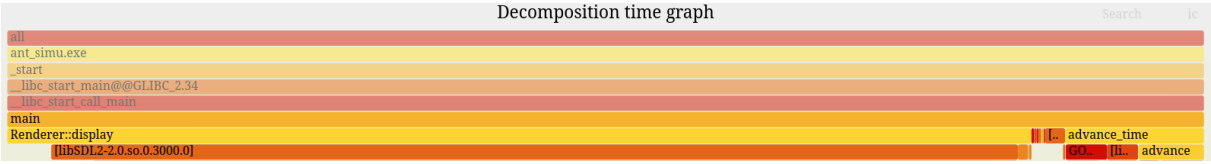

FIGURE 2 – Décomposition du temps d'exécution du programme vectorisé.

L'omniprésence de l'opérateur d'accès (`operator[]`) dans ces graphes confirme que la gestion de la mémoire est le principal frein à la performance, comme illustré dans la Figure 3, justifiant ainsi la transition vers une structure de données vectorisée (*Structure of Arrays*) pour les étapes suivantes.

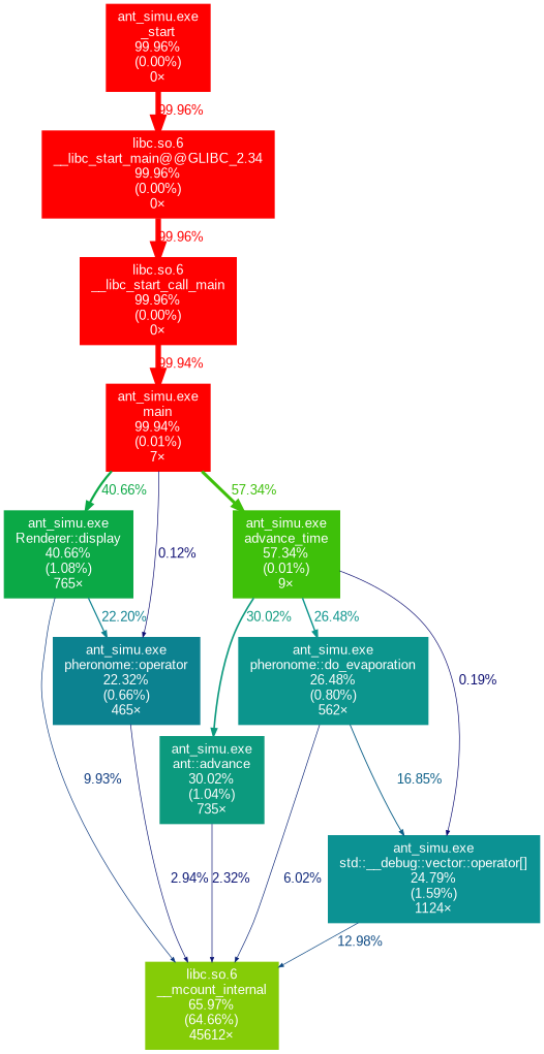

FIGURE 3 – Analyse des accès mémoire et impacts de la structure AoS.

Le graphe d'exécution détaillé du programme vectorisé est présenté dans la Figure 4.

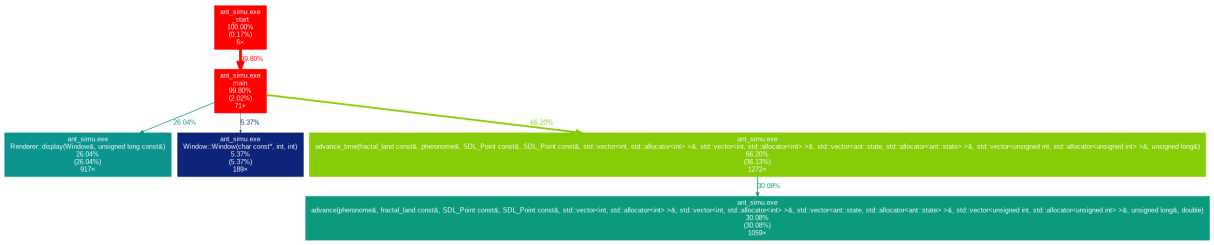

FIGURE 4 – Graph d'execution vectorisé.

Grâce à l'implémentation vectorisée, on observe une réduction significative de la proportion de temps passé dans la fonction la plus coûteuse, à savoir le calcul de la mise à jour des positions des fourmis. Cela montre que le traitement principal est désormais mieux optimisé. Par conséquent, dans de futurs travaux, il serait possible de réduire l'impact de la visualisation, soit en diminuant sa fréquence de mise à jour, soit en la désactivant temporairement, afin de concentrer davantage les ressources processeur sur le calcul pur de la simulation.

Synthèse et Transition vers l'Optimisation en Vectorisation

La modification la plus fondamentale concerne la réorganisation de la façon dont les données des fourmis sont stockées en mémoire. Dans la version classique, chaque fourmi est un objet (`struct ant`) regroupant l'ensemble de ses attributs. Le schéma **AoS**, montré dans la Figure 5, implique que lors de l'itération sur les fourmis, le processeur doit charger des blocs mémoire non-contigus : les champs internes de chaque objet sont entrelacés avec ceux des autres fourmis dans la mémoire physique. Ce phénomène engendre de multiples *cache misses* à chaque accès, saturant ainsi le bus de données - goulot d'étranglement précisément mis en évidence lors de la phase d'analyse. Dans la version vectorisée, les attributs sont séparés en tableaux distincts : `ants_x`, `ants_y`, `ants_state`, `ants_seeds`. Ce schéma **SoA** garantit que lors du traitement d'un attribut particulier pour l'ensemble des fourmis (par exemple, les coordonnées x), les données sont contiguës en mémoire. Le *prefetcher* matériel peut ainsi anticiper les accès successifs, et un même bloc de cache alimente plusieurs itérations consécutives. Il en résulte une amélioration directe de la bande passante de données et une réduction du nombre de cache misses.

Version classique

```
vector<ant> ants;
ants.reserve(nb_ants);

```

Version vectorisée

```
vector<int> ants_x(nb_ants);
vector<int> ants_y(nb_ants);
vector<state> ants_state(nb_ants);
vector<uint> ants_seeds(nb_ants);

```

FIGURE 5 – Comparaison entre versions de la représentation des fourmis.

La restructuration des données en tableaux indépendants rend le boucle principale de mise à jour des fourmis naturellement parallélisable via la directive `#pragma omp parallel for`, comme observée dans la Figure 6.

Version classique

```
for (size_t i=0; i<ants.size();++i)
    ants[i].advance(phen, land,
   pos_food, pos_nest, cpteur);

```

Version vectorisée

```
size_t local_cpteur_food = 0;
size_t seed_copy;
#pragma omp parallel for           \
    reduction(+:local_cpteur_food) \
    private(consumed_time, seed_copy)
for(size_t i= 0; i<nb_ants; ++i){
    // some code...
}
cpteur_food += local_cpteur_food;

```

FIGURE 6 – Utilisation de variables privées pour chaque thread.

Deux aspects sont essentiels dans cette implémentation :

— **La clause** `reduction` : le compteur `cpteur_food` constituerait une variable partagée entre les threads, exposant le code à des *race conditions*. En utilisant `reduction(+:local_cpteur_food)`, chaque thread maintient sa propre copie locale du compteur.

— **La clause** `private` : les variables telles que `consumed_time` et `seed_copy` sont déclarées privées, garantissant que chaque thread opère dans son propre espace de pile sans interférence avec les autres.

Cette approche évite le recours à des sections critiques (`#pragma omp critical`) à l'intérieur de la boucle, lesquelles constitueraient un point de sérialisation annulant une grande partie du gain de parallélisme.

Version classique

```
do {
    npos = old_pos_ant;
    int d = dir_choice();
    if ( d==1 ) npos.x  -= 1;
    if ( d==2 ) npos.y -= 1;
    if ( d==3 ) npos.x  += 1;
    if ( d==4 ) npos.y += 1;
} while (phen[npos][ind_pher] == -1);

```

Version vectorisée

```
do {
    npos = old_pos_ant;
    int d = dir_choice();
    npos.x  += (d==3) - (d==1);
    npos.y  += (d==4) - (d==2);
} while (phen[npos][ind_pher] == -1);

```

FIGURE 7 – Comparaison entre versions de la représentation des fourmis.

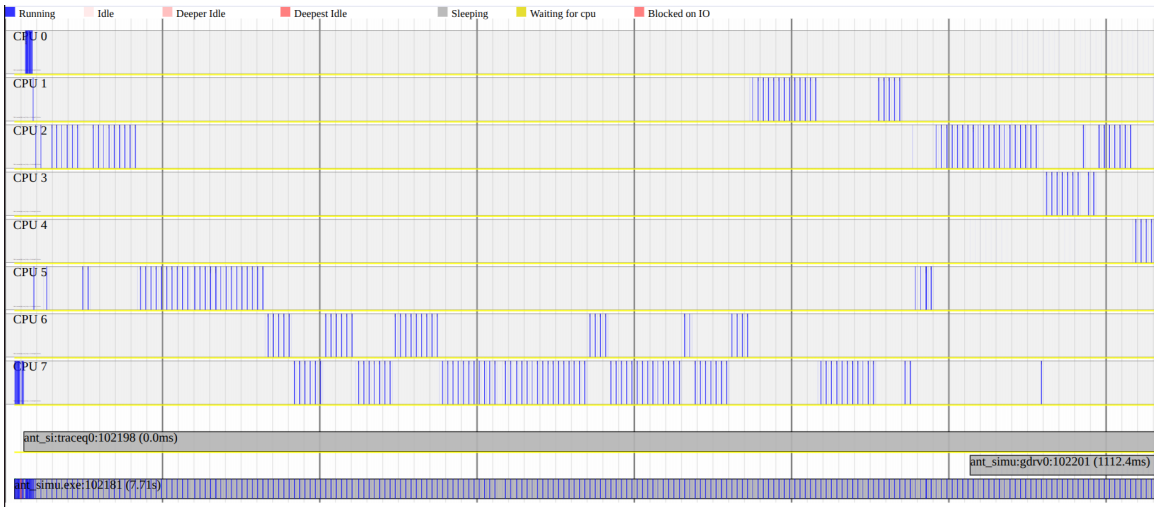

FIGURE 8 – Diagramme d'(sériel) CPU montrant la dispersion de la charge et les changements de contexte.

Synthèse et Transition vers l'optimization en openMP

Dans cette section, nous présentons les optimisations introduites pour améliorer la performance de la simulation de fourmis en utilisant OpenMP. Initialement, La fonction `do_evaporation` maintenant déclare un pointeur `buffer` vers les données des phéromones et utilise `__restrict` pour indiquer au compilateur qu'il n'existe pas de chevauchement de mémoire, permettant ainsi une vectorisation efficace. La directive `#pragma omp parallel for` distribue les lignes du tableau entre les threads disponibles. À l'intérieur, les boucles sur les colonnes sont annotées avec `#pragma GCC ivdep` et `#pragma GCC unroll 4`, ce qui informe le compilateur que les itérations sont indépendantes et peut être déroulées pour les unités SIMD, augmentant le parallélisme au niveau des vecteurs.

```
void do_evaporation() {
    const std::size_t dim = m_dim;
    const double beta = m_beta;
    auto* __restrict buffer = m_buffer_pheronome.data();
    #pragma omp parallel for
    for (std::size_t i = 1; i <= dim; ++i) {
        const std::size_t row_start = i * m_stride + 1;
        #pragma GCC ivdep
        #pragma GCC unroll 4
        for (std::size_t j = 0; j < dim; ++j) {
            std::size_t idx = row_start + j;
            buffer[idx][0] *= beta;
            buffer[idx][1] *= beta;
        }
    }
}

```

FIGURE 9 – Parallel evaporation step.

Concernant la fonction `advance` qui met à jour la position et l'état des fourmis, l'optimisation OpenMP a été introduite pour exploiter le parallélisme au niveau des fourmis. Chaque thread maintient un compteur local de nourriture `thread_local_cpteur` afin d'éviter les conflits d'écriture lors de l'incrémentation globale. La boucle principale sur toutes les fourmis est parallé-

lisée avec `#pragma omp for` et une réduction sur `local_cpteur_food` est appliquée, permettant d'agréger correctement les contributions de chaque thread à la quantité totale de nourriture collectée.

```
#pragma omp parallel reduction(+:local_cpteur_food)
{
    std::size_t thread_local_cpteur = 0;
#pragma omp for
    for(size_t i = 0; i < nb_ants; ++i)
    {
        // ... computation using thread_local_cpteur ...
    }
}

```

FIGURE 10 – Extrait de la fonction `advance`.

L'analyse de profilage après l'optimisation OpenMP montre que le principal coût d'exécution provient désormais du rendu graphique,comme illustré dans la Figure 11.

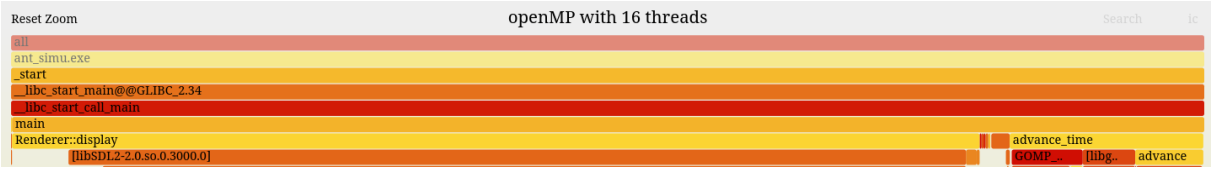

FIGURE 11 – Participation de chaque fonction.

La Figure 12 montre l'utilisation des cœurs CPU lors de l'exécution parallèle. On observe la répartition du travail entre les différents threads ainsi que plusieurs changements de contexte pendant l'exécution.

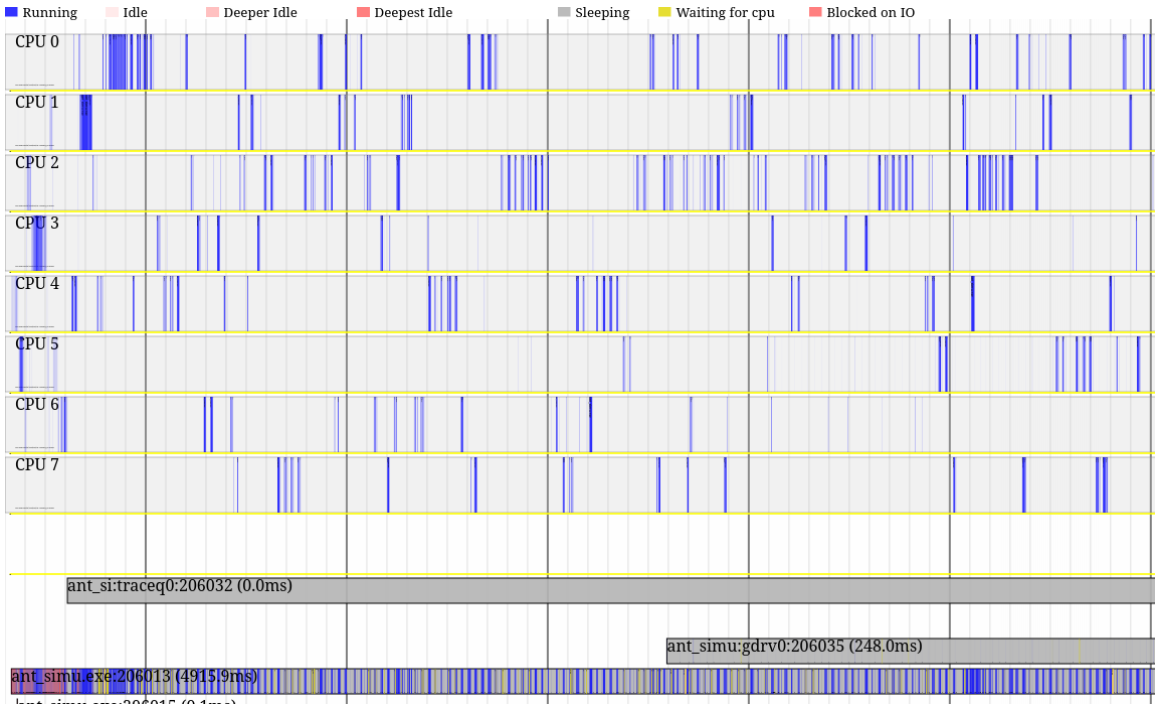

FIGURE 12 – Diagramme d'exécution parallèle du CPU illustrant la répartition de la charge et les changements de contexte.

4. MPI - implementation

Dans cette simulation, l'idée centrale est de découper le problème en deux dimensions : les fourmis elles-mêmes et le plateau de phéromones. Chaque rang MPI se voit attribuer un sous-ensemble de fourmis à gérer et une bande spécifique de lignes du plateau pour appliquer l'évaporation. Cette double division garantit que chaque rang peut travailler de manière autonome sur ses données locales tout en limitant les communications nécessaires entre les processus.

La logique de répartition des fourmis repose sur un découpage simple mais efficace : le nombre total de fourmis est divisé proportionnellement au nombre de rangs MPI. Chaque rang obtient donc un intervalle contigu de fourmis dont il est responsable, défini par les indices [local_ant_begin, local_ant_end[.

```
size_t dim = land.dimensions();
size_t local_ant_begin =  mpi_rank      * nb_ants / mpi_size;
size_t local_ant_end   = (mpi_rank + 1) * nb_ants / mpi_size;
size_t evap_begin = mpi_rank       * dim / mpi_size;
size_t evap_end   = (mpi_rank + 1) * dim / mpi_size;
```

FIGURE 13 – Répartition parallèle des fourmis et des plages d'évaporation entre les rangs MPI.

Lors de l'étape de déplacement, chaque rang ne met à jour que les positions et états de ses propres fourmis, ce qui évite tout conflit d'écriture et permet d'utiliser des parallélismes internes comme OpenMP pour accélérer le calcul.

```
MPI_Allreduce(MPI_IN_PLACE,
              phen.buffer_data(),
              static_cast<int>(total),
              MPI_DOUBLE,
              MPI_MIN,
              MPI_COMM_WORLD);
}
```

FIGURE 14 – MPI reduction pour propager les valeurs evaporées.

Pour les phéromones, une stratégie similaire est appliquée sur le plateau. Chaque rang se voit assigner une bande de lignes [evap_begin, evap_end[pour laquelle il appliquera l'évaporation localement. Une fois l'évaporation locale effectuée sur la bande assignée, un second MPI_Allreduce avec l'opération MIN est utilisé pour propager les valeurs évaporées sur l'ensemble du plateau, assurant que toutes les cellules reflètent correctement la diminution de phéromone, tout en préservant les cellules spéciales comme le nid et la nourriture.

À l'intérieur de cette tranche, une boucle parallèle OpenMP itère sur chaque fourmi locale. Chaque thread possède sa copie privée de la graine aléatoire (`seed_copy`) afin d'assurer que les tirages aléatoires sont indépendants et reproductibles, même en parallèle. L'usage de la réduction sur `local_cpteur_food` permet de compter correctement la nourriture collectée sans sections critiques coûteuses.

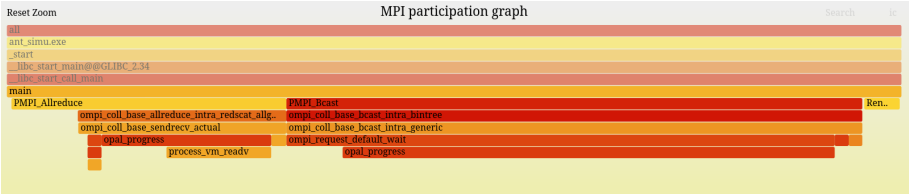

FIGURE 16 – Participation de chaque fonction en utilisant MPI

```
        std::size_t seed_copy=0;
        double consumed_time = 0.;
        std::size_t local_count = local_end - local_begin;
        std::size_t local_cpteur_food = 0;
        #pragma omp parallel for reduction(+:local_cpteur_food) private(
    consumed_time, seed_copy)
        for(size_t k = 0; k < local_count; ++k) {
            size_t i = local_begin + k;
```

FIGURE 15 – MPI implementation de la fonction advance

Le déplacement de chaque fourmi suit la logique originale : si le choix aléatoire dépasse `m_eps` ou si aucun phéromone n'est détecté, la fourmi se déplace aléatoirement ; sinon, elle suit la direction maximale de phéromone. Les positions et états des fourmis sont mis à jour à chaque pas de temps, et la graine aléatoire est sauvegardée après chaque itération pour garantir la continuité des tirages.

Cette approche combine distribution MPI pour la scalabilité sur plusieurs processus et OpenMP pour exploiter le parallélisme sur les coeurs locaux, tout en conservant la cohérence des données partagées et des générateurs aléatoires.

5. Analyse du temps

Afin d'évaluer l'impact des différentes optimisations introduites dans ce travail, nous avons mesuré le temps d'exécution du programme pour plusieurs versions de l'implémentation.

Les expériences ont été réalisées en exécutant plusieurs itérations du programme afin de réduire les fluctuations liées au système d'exploitation et aux accès mémoire. Les résultats présentés correspondent au temps moyen d'exécution mesuré pour chaque configuration.

Le Tableau 1 résume les temps obtenus pour les différentes versions du programme ainsi que le speedup calculé par rapport à la version séquentielle.

Description	**# coeurs**	**Temps (ms)**	**Speedup**
Séquentiel	1	57.10	1.00
Vectorisation	1	33.34	1.71
openMP	2	169.69	0.34
openMP	4	196.42	0.29
openMP	8	51.01	1.12
openMP	16	43.15	1.32
MPI+openMP	16	7924.50	0.01

TABLE 1 – Mesures du temps d'exécution pour les différentes versions du programme.

Serial

La version séquentielle constitue la référence de performance du programme. Dans cette configuration, l'ensemble des calculs est exécuté sur un seul cœur sans aucune forme de parallélisme.

Les mesures obtenues montrent un temps d'exécution moyen de 57.10 ms, avec des valeurs comprises entre 23.22 ms et 2004.30 ms. Cette variabilité peut être expliquée par la présence de défauts de page ainsi que par des accès mémoire peu efficaces liés à la structure de données initiale.

Cette version met en évidence les principaux goulets d'étranglement du programme, notamment dans les fonctions responsables de la simulation des fourmis et de la gestion des phéromones.

Vectorization

La vectorisation vise à améliorer l'efficacité des accès mémoire en transformant la structure des données de type *Array of Structures* (AoS) vers une organisation *Structure of Arrays* (SoA).

Cette transformation améliore la localité mémoire et permet au processeur d'exploiter plus efficacement le cache et le préchargement des données.

Les résultats obtenus montrent une amélioration significative des performances. Le temps d'exécution moyen passe de 57.10 ms pour la version séquentielle à 33.34 ms après vectorisation.

Cela correspond à un **speedup d'environ 1.71**, ce qui confirme que les accès mémoire représentaient un facteur limitant majeur dans l'implémentation initiale.

OpenMP

Afin d'exploiter le parallélisme sur mémoire partagée, la boucle principale de mise à jour des fourmis a été parallélisée à l'aide d'OpenMP.

Les performances ont été évaluées pour différentes configurations de threads. Les résultats montrent que l'utilisation d'un faible nombre de threads (2 ou 4) introduit un surcoût important lié à la gestion des threads et aux synchronisations. Dans ces cas, le temps d'exécution moyen devient supérieur à celui de la version séquentielle.

En revanche, lorsque le nombre de threads augmente, les performances s'améliorent. Avec 8 threads, le temps d'exécution moyen descend à 51.01 ms, ce qui correspond à un speedup de 1.12. Avec 16 threads, le temps atteint 43.15 ms, soit un speedup d'environ 1.32.

Ces résultats montrent que la parallélisation OpenMP devient réellement efficace lorsque le nombre de threads est suffisamment élevé pour compenser le coût de création et de synchronisation des threads.

MPI + OpenMP

Une version hybride combinant MPI et OpenMP a également été évaluée afin d'exploiter le parallélisme distribué. Cependant, les résultats montrent un speedup très faible.

Cela s'explique principalement par la taille relativement réduite du problème. Dans ce cas, le coût des communications et des synchronisations entre les processus MPI devient dominant par rapport au temps de calcul.

En pratique, les approches hybrides MPI + OpenMP sont généralement plus adaptées à des simulations de grande échelle exécutées sur des clusters ou des supercalculateurs, où la charge

de calcul est suffisamment importante pour compenser le coût des communications.

6. Configuration machine

Les expériences ont été réalisées sur un processeur Intel Core i5-11300H (11e génération) doté de 4 cœurs physiques et 8 threads grâce à l'Hyper-Threading. Les principales caractéristiques du matériel utilisé sont résumées dans le Tableau 2.

Table 2 – Caractéristiques du processeur Intel Core i5-11300H pertinentes pour le parallélisme

[H] **Caractéristiques**	**Spécification**
Modèle	Intel Core i5-11300H (11e génération)
Microarchitecture	Tiger Lake
Coeurs physiques / threads	4 coeurs / 8 threads (Hyper-Threading)
Fréquence de base	3,1 GHz
Fréquence turbo maximale	4,4 GHz
Extensions SIMD	AVX2, AVX-512F, AVX-512BW, AVX-512VL, AVX-512VNNI, AVX-512BITALG
Autres instructions pertinentes	AES, SHA, RdRand, RDSEED, MOVDIRI, MOVDIR64B, FSGSBASE
Technologies de parallélisme	Hyper-Threading, Turbo Boost
Prise en charge 64 bits	Oui